INFORME DE ACCESIBILIDAD WEB

Proyecto React

Alumno: Sandro Pegoraro
Curso: 2º DAW
Módulo: Diseño de Interfaces Web
Proyecto: Mis Series Imprescindibles

---

1. ENLACES DEL PROYECTO

Despliegue en Netlify: https://podcastverificacion.netlify.app

Repositorio GitHub: https://github.com/sandrow488/PodCast-Serie-Verificacion.git

En el repositorio se puede consultar el historial de commits donde se refleja toda la evolución del proyecto. Cada commit corresponde a un cambio concreto y documentado, lo que permite ver de forma clara qué archivos se modificaron en cada fase y por qué motivo.

-

2. INTRODUCCIÓN Y OBJETIVOS

Este informe documenta el proceso de auditoría y mejora de accesibilidad web realizado sobre el proyecto "Mis Series Imprescindibles", una aplicación web desarrollada con React. El objetivo principal era conseguir que la web cumpliera con el nivel de conformidad AA de las pautas WCAG 2.2 (Web Content Accessibility Guidelines), que es el nivel que exige la legislación europea para sitios web públicos.

La metodología seguida ha sido iterativa: primero analicé el estado inicial de la web con herramientas automáticas, después identifiqué los fallos manualmente, apliqué las correcciones correspondientes y finalmente volví a auditar para comprobar la mejora. Las herramientas que he utilizado para la auditoría son las siguientes:

- **Lighthouse (Chrome DevTools):** auditoría automática integrada en el navegador que puntúa la accesibilidad de 0 a 100.
- **WAVE (Web Accessibility Evaluation Tool):** extensión que señala visualmente los errores y alertas de accesibilidad directamente sobre la página.
- **Axe DevTools:** extensión de Deque que detecta violaciones de WCAG y las clasifica por gravedad (critical, serious, moderate, minor).
- **Pruebas manuales de teclado:** navegación completa por la web usando exclusivamente Tab, Shift+Tab y Enter para verificar que todos los elementos interactivos son alcanzables y operables.

3. ESTRUCTURA Y TECNOLOGÍAS

La aplicación está construida con React 18 y empaquetada con Vite. El enrutamiento se gestiona con React Router DOM, lo que hace que sea una SPA (Single Page Application): el contenido se carga dinámicamente sin recargar la página completa. Esto da una experiencia de usuario muy fluida, pero genera problemas de accesibilidad específicos, ya que los lectores de pantalla no siempre detectan los cambios de contenido que ocurren sin recarga.

Para los estilos he utilizado Tailwind CSS, un framework de utilidades que permite aplicar estilos directamente en las clases del HTML. La web está desplegada en Netlify con un archivo `_redirects` configurado para que todas las rutas de React Router funcionen correctamente en producción (sin esto, al recargar cualquier ruta que no sea la raíz, Netlify devuelve un 404).

La web tiene las siguientes páginas y componentes principales:

- **Home.jsx:** página de inicio con el hero, el catálogo de episodios y la sección de promoción.
- **EpisodeDetails.jsx:** vista detallada de cada episodio con reproductor de audio y transcripción.
- **Contact.jsx:** formulario de contacto con validación en tiempo real.
- **InformePDF.jsx:** este mismo informe, que se puede descargar como PDF.
- **App.jsx:** componente raíz con el enrutamiento, el header, el footer y el skip link.

4. TESTEO INICIAL

Puntuación inicial en Lighthouse: 71/100

Al pasar Lighthouse por primera vez sobre la web, obtuve una puntuación de 71 en accesibilidad. El informe de Lighthouse señaló varios errores concretos: elementos de imagen sin atributo alt, contraste insuficiente entre texto y fondo en varias zonas, inputs de formulario sin label asociado, y falta de landmark regions en el HTML. WAVE confirmó estos resultados y además detectó que no existía un encabezado H1 en algunas páginas y que había enlaces vacíos sin texto descriptivo.

La prueba manual de teclado reveló un problema grave: el indicador de foco (focus outline) era prácticamente invisible en la mayoría de elementos interactivos. Esto significa que un usuario que navegue solo con teclado no puede saber en qué botón o enlace está posicionado en cada momento, incumpliendo el criterio 2.4.7 (Focus Visible) de WCAG.

5. FALLOS ENCONTRADOS Y CRITERIOS WCAG INCUMPLIDOS

A continuación detallo los fallos que encontré y el criterio WCAG concreto que incumplía cada uno:

- Uso incorrecto de HTML semántico

La estructura de la página usaba casi exclusivamente elementos div genéricos. No había etiquetas semánticas como header, nav, main o footer. Esto impide que los lectores de pantalla identifiquen las diferentes regiones de la página y ofrezcan navegación por landmarks al usuario.

- Imágenes sin texto alternativo

Las imágenes de las portadas de los episodios no tenían el atributo alt, o lo tenían vacío. Sin este atributo, una persona ciega que use un lector de pantalla no recibe ninguna información sobre el contenido de la imagen, solo escucha "imagen" o directamente el nombre del archivo.

- Contraste insuficiente

Varios textos de la web tenían un ratio de contraste por debajo de 4.5:1, que es el mínimo que exige el nivel AA para texto normal. Esto afectaba sobre todo a textos gris claro sobre fondo blanco y a algunos botones con colores de fondo que no contrastaban lo suficiente con el texto.

- Formulario sin labels vinculados

Los campos del formulario de contacto (nombre, email, asunto, mensaje) no estaban vinculados a sus respectivas etiquetas label mediante el atributo htmlFor/id. Esto hacía que los lectores de pantalla no anunciaran qué dato se esperaba en cada campo, lo que dificulta mucho su uso para personas con discapacidad visual.

- Foco invisible

El outline de foco por defecto del navegador estaba eliminado con CSS (outline: none) sin ofrecer un estilo alternativo visible. Esto hacía imposible navegar con teclado de forma efectiva, ya que el usuario no puede ver qué elemento tiene el foco en cada momento.

- Sin mecanismo de salto

No existía ningún enlace de tipo "skip to content" que permitiera a los usuarios de teclado saltarse el bloque de navegación repetido y acceder directamente al contenido principal de cada página.

6. SOLUCIONES APLICADAS

- HTML semántico

Sustituí los div genéricos por etiquetas semánticas: la cabecera ahora usa header, el menú de navegación está dentro de un nav con aria-label="Navegación principal", el contenido de cada página se envuelve en un main con id="main-content", y el pie de página usa footer. También corregí la jerarquía de encabezados para que cada página tenga un solo H1 y el resto sigan un orden lógico (H2, H3...).

- Textos alternativos en imágenes

Añadí atributos alt descriptivos a todas las imágenes del catálogo. Por ejemplo, en lugar de dejar alt vacío, ahora pone "Portada del episodio: Breaking Bad" o "Portada del episodio: Stranger Things". De esta forma el lector de pantalla transmite información útil sobre el contenido visual.

- Corrección de contraste

Cambié la paleta de colores principal a tonos indigo oscuro (indigo-800 / indigo-950) sobre fondos blancos, consiguiendo ratios de contraste superiores a 7:1 en todos los textos importantes. Los textos secundarios se cambiaron a slate-700 sobre blanco, con un ratio de 5.3:1, que cumple sobradamente el mínimo de 4.5:1.

- Formulario accesible

Vinculé cada input con su label usando htmlFor y id. Añadí atributos aria-required="true" a los campos obligatorios, aria-describedby para asociar los mensajes de error con su campo correspondiente, y un div con aria-live="assertive" que anuncia automáticamente cuando hay errores en el formulario. De esta manera, cuando un usuario con lector de pantalla envía el formulario con errores, escucha inmediatamente cuántos errores hay y puede corregirlos.

- Foco visible personalizado

Implementé estilos de foco visibles en todos los elementos interactivos usando las utilidades focus:outline-none focus:ring-2 focus:ring-white (en el header) y focus:ring-indigo-500 (en el cuerpo). De esta forma, al navegar con Tab, aparece un anillo de color bien visible alrededor de cada enlace, botón o campo de formulario.

- Skip Link

Añadí un enlace oculto al principio del DOM que apunta a #main-content. Este enlace es invisible visualmente pero aparece cuando recibe el foco con teclado (usando las

clases sr-only y focus:not-sr-only de Tailwind). Al pulsarlo, el foco salta directamente al contenido principal de la página, evitando tener que pasar por todos los enlaces del menú.

- Reducción de movimiento

En el archivo index.css añadí una media query @media (prefers-reduced-motion: reduce) que desactiva todas las transiciones y animaciones CSS cuando el usuario tiene activada la opción de "reducir movimiento" en su sistema operativo. Esto cumple con el criterio 2.3.3 de WCAG y beneficia a personas con trastornos vestibulares o epilepsia fotosensible.

7. COMPROBACIÓN FINAL

Después de aplicar todas las correcciones anteriores y hacer los commits correspondientes, volví a ejecutar las herramientas de auditoría para verificar las mejoras.

Puntuación final en Lighthouse: 97/100

La puntuación de accesibilidad subió de 71 a 97 puntos. WAVE ya no muestra errores (solo algunas alertas informativas que no son violaciones). Axe DevTools tampoco detecta violaciones de ningún nivel. La navegación por teclado funciona correctamente en todas las páginas: el foco es visible, el skip link funciona, y los mensajes de error del formulario se anuncian por el lector de pantalla.

Los 3 puntos que faltan para llegar a 100 en Lighthouse se deben a que algunas imágenes externas no tienen un tamaño explícito definido en el HTML (width/height), lo que Lighthouse considera una buena práctica pero no es un error de accesibilidad en sí mismo.

8. CONCLUSIÓN

El proyecto ha pasado por una transformación significativa desde su estado inicial hasta la versión final. Cuando empecé, la web era funcional pero no tenía en cuenta a los usuarios con diversidad funcional: no había textos alternativos en las imágenes, la estructura del HTML no estaba organizada con etiquetas semánticas, el formulario de contacto no estaba correctamente etiquetado y era inutilizable con un lector de pantalla, los colores no cumplían con los requisitos mínimos de contraste, y la navegación por teclado era imposible porque no se veía el foco.

Durante el proceso de mejora he aprendido que la accesibilidad web no es simplemente añadir unos cuantos atributos ARIA al código, sino que requiere un enfoque integral que abarca la estructura del HTML, los estilos CSS, la gestión del foco, y el comportamiento dinámico de la aplicación. Cada cambio que hice estaba motivado por un criterio concreto de las WCAG 2.2, como el 1.1.1 para los textos alternativos, el 1.3.1 para la semántica, el 1.4.3 para el contraste, el 2.4.1 para el skip link, o el 2.4.7 para el foco visible.

Uno de los aprendizajes más importantes ha sido entender los problemas específicos que genera una SPA en términos de accesibilidad: al no recargar la página completa, los lectores de pantalla pueden perder el contexto cuando cambia el contenido dinámicamente. Para resolver esto fue clave el uso de aria-live en los mensajes de error del formulario y la correcta gestión del foco al cambiar de ruta.

El resultado final es una aplicación con una puntuación de 97/100 en Lighthouse que cumple con los criterios de nivel AA de WCAG 2.2. De este trabajo me quedo con la idea de que la accesibilidad debería tenerse en cuenta desde el inicio del desarrollo, no como algo que se añade al final, ya que es mucho más costoso y complicado corregir los problemas a posteriori que diseñar con accesibilidad en mente desde el primer momento.